Introduction

The purpose of this document is to evaluate potential security weaknesses and implement security controls within the mobile app's infrastructure.

Scope of Work

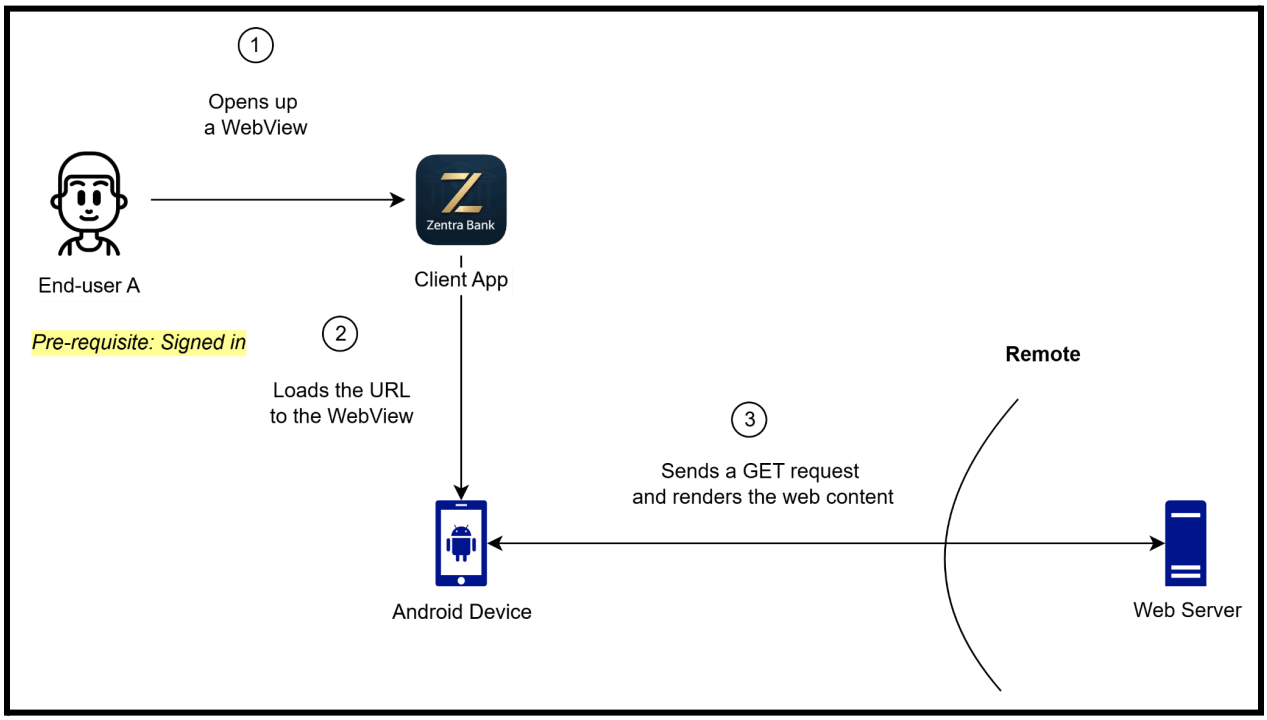
This exercise focuses on 1 user story within the Zentra Bank mobile app:

1. As a user, I want to **view web content through a web view** so that I don't have to switch to an external browser.

Out of Scope

This exercise strictly focuses on the mobile application and its functionalities. Any components outside the mobile app, which includes but not limited to REST APIs, server infrastructure, remote database, and external services, are not within the scope.

User story 7



The data flow for this user story is as follows:

"As a user, I want to view web content through a web view so that I don't have to switch to an external browser."

1. The **End-user** opens up a WebView
2. The **Client App (CA)** loads the URL to the WebView (i.e. csa.gov.sg)
3. The **Android Operating System (AOS)** WebView sends a GET request and renders the web content in the WebView

▼ Request Headers

:authority	www.csa.gov.sg
:method	GET
:path	/
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9
Cache-Control	max-age=0
Cookie	_sp_ses.3ab5=*; _sp_id.3ab5=b537e8be-ea4f-4e40-94e5-241e2622a072.1759833017.2.1759985940.1759833020.2a01a1b3-5465-48cc-853f-ea45483df6ca.426e9b31-b641-4569-8ad9-967082eba59b...0; _ga_5KSX69ZFL1=GS2.1.s1759985941$o2$g0$t1759985941$j60$l0$h0; _clck=8844f4%5E2%5Eg00%5E0%5E2106; _ga=GA1.3.558245804.1759833019; _gid=GA1.3.76484450.1759985942; _gat_gtag_UA_216510461_1=1; _clsk=1qzodgp%5E1759985943417%5E1%5E0%5Ev.clarity.ms%2Fcollect
If-Modified-Since	Wed, 08 Oct 2025 10:14:26 GMT
If-None-Match	W/"3219ae5d98228d3e4d5540e917f15c9a"
Priority	u=0, i
Sec-Ch-Ua	"Chromium";v="140", "Not=A?Brand";v="24", "Android WebView";v="140"
Sec-Ch-Ua-Mobile	?1
Sec-Ch-Ua-Platform	"Android"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	same-origin
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Linux; Android 10; Android SDK built for x86 Build/QSR1.211112.011; wv) AppleWebKit/537.36 (KHTML, like Gecko) Version/4.0 Chrome/140.0.7339.207 Mobile Safari/537.36
X-Requested-With	com.csec.zentra

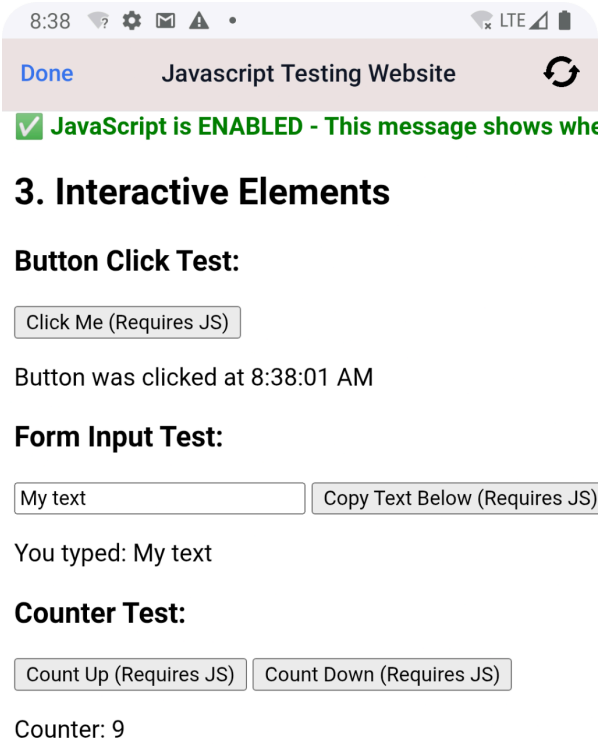
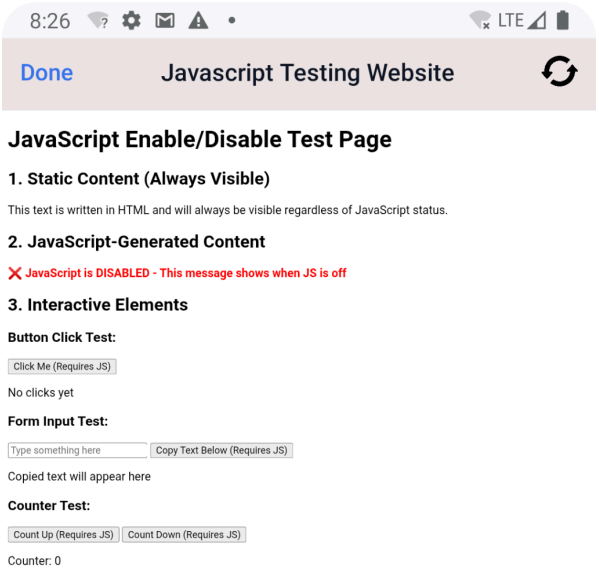
▼ Response Headers

Age	456
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=600
Content-Security-Policy	default-src 'none';base-uri 'self';script-src 'self' 'unsafe-inline' blob: https://api.search.gov.sg https://www.search.gov.sg https://*.wogaa.sg https://www.google-analytics.com https://www.googletagmanager.com https://connect.facebook.net https://graph.facebook.com https://*.licdn.com https://www.youtube.com https://webchat.vica.gov.sg https://webchat.mol-vica.com https://*.clarity.ms https://*.ask.gov.sg ;style-src 'self' 'unsafe-inline' https://assets.wogaa.sg https://www.search.gov.sg https://webchat.vica.gov.sg https://webchat.mol-vica.com ;object-src 'none' ;connect-src 'self' https://isomer-user-content.by.gov.sg https://browser-intake-datadoghq.com https://api.search.gov.sg https://*.wogaa.sg https://api-chat-fe-flag.vica.gov.sg https://chat.vica.gov.sg wss://chat.vica.gov.sg https://api-chat-fe-flag.mol-vica.com https://chat.mol-vica.com wss://chat.mol-vica.com https://www.google-analytics.com https://www.google.com https://analytics.google.com https://stats.g.doubleclick.net https://px.ads.linkedin.com https://*.clarity.ms https://c.bing.com https://ask.gov.sg https://*.ask.gov.sg https://data.gov.sg https://*.data.gov.sg ;font-src 'self' data: https://fonts.gstatic.com https://www.search.gov.sg https://assets.wogaa.sg ;frame-src 'self' https://www.search.gov.sg https://www.google.com https://www.googletagmanager.com https://td.doubleclick.net https://www.onemap.gov.sg https://www.youtube-nocookie.com https://player.vimeo.com https://m.facebook.com https://www.facebook.com https://docs.google.com https://form.gov.sg https://maps.gov.sg ;img-src 'self' https: ;manifest-src 'self' ;media-src 'self' ;worker-src 'self' ;frame-ancestors 'self' ;
Date	Thu, 09 Oct 2025 04:52:15 GMT
Etag	W/"3219ae5d98228d3e4d5540e917f15c9a"
Last-Modified	Wed, 08 Oct 2025 10:14:26 GMT
Referrer-Policy	strict-origin-when-cross-origin
Server	AmazonS3
Strict-Transport-Security	max-age=31536000
Vary	accept-encoding
Vary	Origin
Via	1.1 b93539311f65854b91591e3ebce387d2.cloudfront.net (CloudFront)
X-Amz-Cf-Id	qDGhw744kibVuqJXjLLLYhjWHkXinp37CW9CAh1w_2jQ-e5iEQ_-iw==
X-Amz-Cf-Pop	SIN2-P6
X-Amz-Server-Side-Encryption	AES256
X-Amz-Version-Id	bphrGCsNTVwhWYgx_6sc9wIfe1s3GOvC
X-Cache	Hit from cloudfront
X-Content-Type-Options	nosniff
X-Frame-Options	SAMEORIGIN
X-Xss-Protection	1; mode=block

WebView (FAQ)

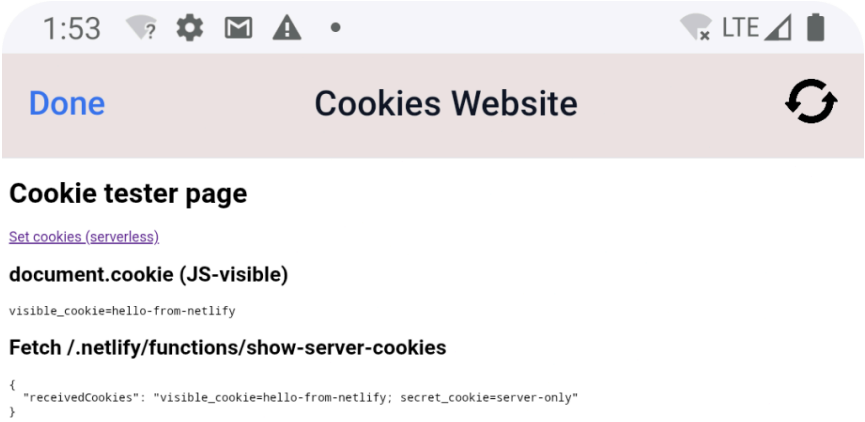
What happens when JavaScript is disabled?

When JavaScript is disabled. Observe that while the html page can still be rendered, JavaScript features like allowing buttons to be clicked, will not work.

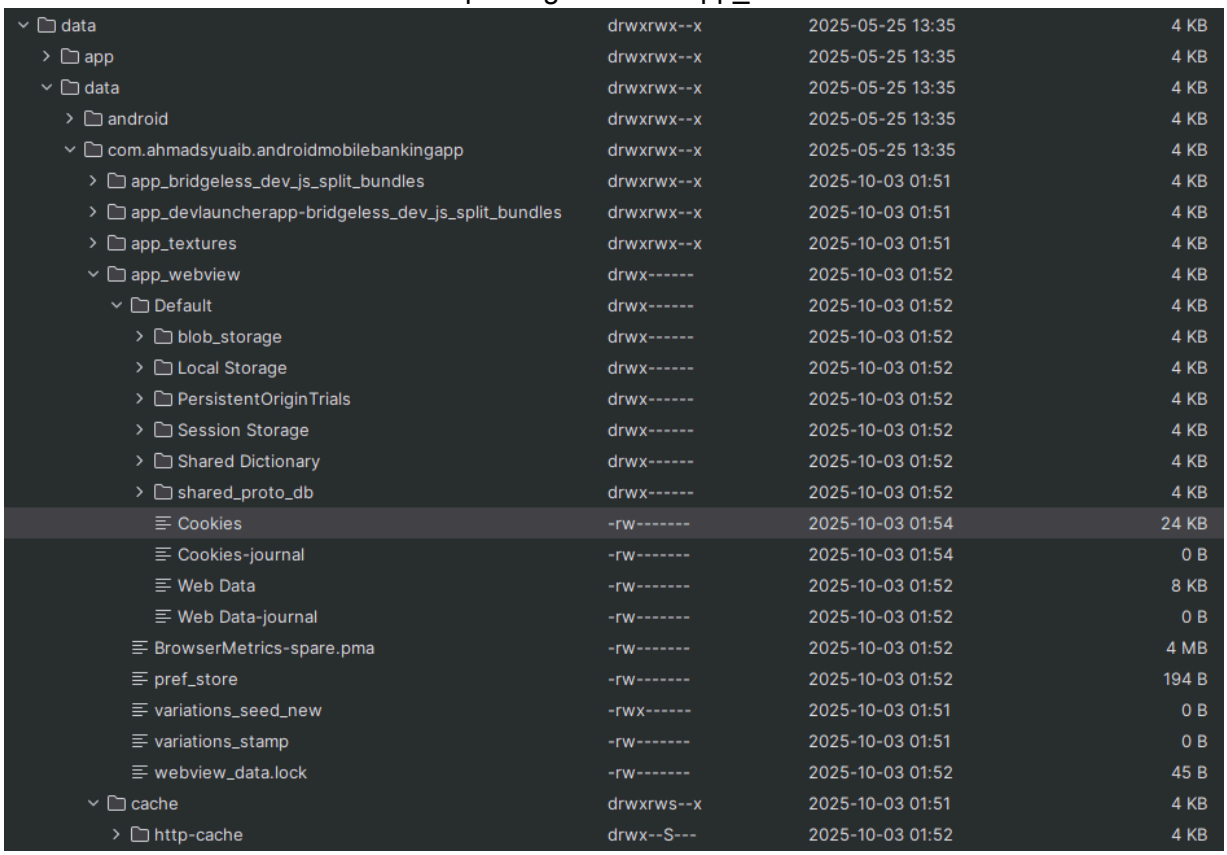
JavaScript enabled.	JavaScript disabled.
	

Does WebView use cookies?

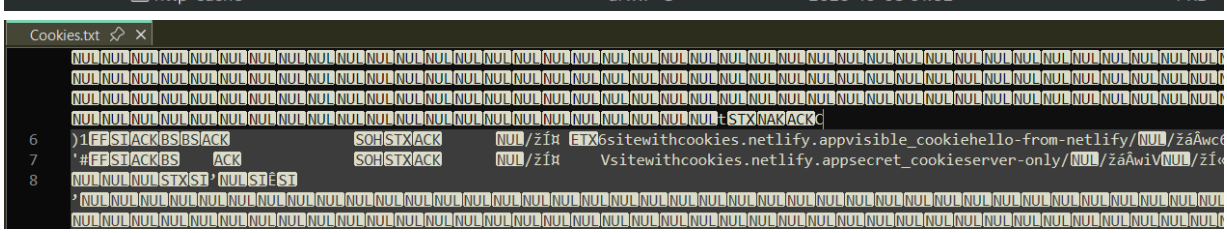
WebView uses cookies, and it's stored within the app's internal storage. Observe that cookies are being stored within the app's internal storage directory.



Cookies file is stored in /data/data/<package-name>/app_webview/Default/cookies



data	drwxrwx--x	2025-05-25 13:35	4 KB
app	drwxrwx--x	2025-05-25 13:35	4 KB
data	drwxrwx--x	2025-05-25 13:35	4 KB
android	drwxrwx--x	2025-05-25 13:35	4 KB
com.ahmadsyuaib.androidmobilebankingapp	drwxrwx--x	2025-05-25 13:35	4 KB
app_bridgeless_dev_js_split_bundles	drwxrwx--x	2025-10-03 01:51	4 KB
app_devlauncherapp-bridgeless_dev_js_split_bundles	drwxrwx--x	2025-10-03 01:51	4 KB
app_textures	drwxrwx--x	2025-10-03 01:51	4 KB
app_webview	drwx------	2025-10-03 01:52	4 KB
Default	drwx------	2025-10-03 01:52	4 KB
blob_storage	drwx------	2025-10-03 01:52	4 KB
Local Storage	drwx------	2025-10-03 01:52	4 KB
PersistentOriginTrials	drwx------	2025-10-03 01:52	4 KB
Session Storage	drwx------	2025-10-03 01:52	4 KB
Shared Dictionary	drwx------	2025-10-03 01:52	4 KB
shared_proto_db	drwx------	2025-10-03 01:52	4 KB
Cookies	-rw-------	2025-10-03 01:54	24 KB
Cookies-journal	-rw-------	2025-10-03 01:54	0 B
Web Data	-rw-------	2025-10-03 01:52	8 KB
Web Data-journal	-rw-------	2025-10-03 01:52	0 B
BrowserMetrics-spare.pma	-rw-------	2025-10-03 01:52	4 MB
pref_store	-rw-------	2025-10-03 01:52	194 B
variations_seed_new	-rwx------	2025-10-03 01:51	0 B
variations_stamp	-rw-------	2025-10-03 01:51	0 B
webview_data.lock	-rw-------	2025-10-03 01:52	45 B
cache	drwxrws--x	2025-10-03 01:51	4 KB
http-cache	drwx--S---	2025-10-03 01:52	4 KB



Methodology

The methodology for this exercise follows a structured approach:

1. Create data flow diagrams to map how data moves within the mobile app
2. Use security testing tools and threat modeling to identify vulnerabilities
    a. Mobile Application Security Framework (MobSF)

3. Evaluate the impact of the identified vulnerability, how they could affect the app
4. Research remediation measures and show their effectiveness

Findings Overview

US07-MOBSF01. My app's WebView enabled debugging. Attackers could use remote webview debugging to reveal hidden files.....9
US07-MOBSF01-C01. Remove threat by ensuring the app sets WebView Remote Debugging to false.....12
US07-MOBSF01-C02. Remove threat by ensuring the app set allowFileAccessFromFileURLs webview setting to false.....13
US07-MOBSF02. My app writes to external storage. Attackers could read the files created in external storage.....15
US07-MOBSF02-C01. Remove threat by updating target SDK to Android 11 to enforce scoped storage, disallowing other apps from accessing private files.....16

Template

Legend
- White background cells: For interns to fill
- Blue / Orange / Green background cells: For Supervising officer to fill

Issue: App can be installed on a vulnerable unpatched Android version
Issue ID: US01-MOBSF01. My app allows installation by devices running on Android 7.0 and above. Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit. **Cause**: My app allows installation by devices running on Android 7.0 and above. **Effect**: Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit.
Demonstration: - Show what this issue means to the App. - Explain with evidence on what this issue means to the App?
Control ID: US01-MOBSF01-C01. Remove threat by ensuring the app installs only on devices running Android versions with active security support. **Control**: Remove threat by ensuring the app installs only on devices running Android versions with active security support.
Implemented at: Client App / Authentication Server / Database **Demonstration**: - Show how the control mitigates the threat. - Explain with evidence on how you implement the control?
Status: tried (implemented) / tested (tested for effectiveness)
Another implementation

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)

Control ID:
US01-MOBSF01-C02. Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Control:
Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)

New risks that arise as a result of implementing user stories

Issue: Remote WebView debugging is enabled

Issue ID:
US07-MOBSF01. My app's WebView enabled debugging. Attackers could use remote webview debugging to reveal hidden files.

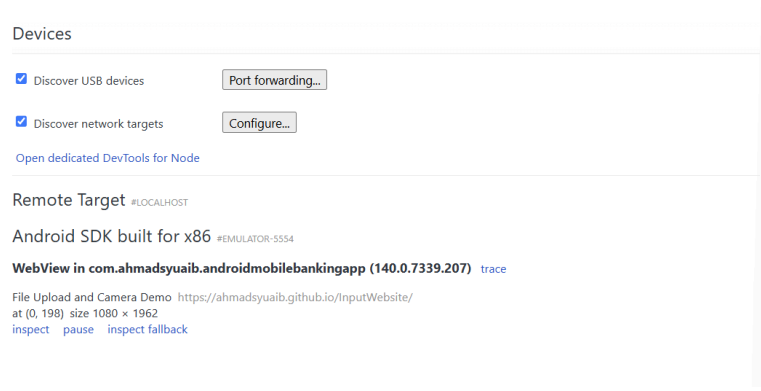
Cause:
My app's WebView enabled debugging.

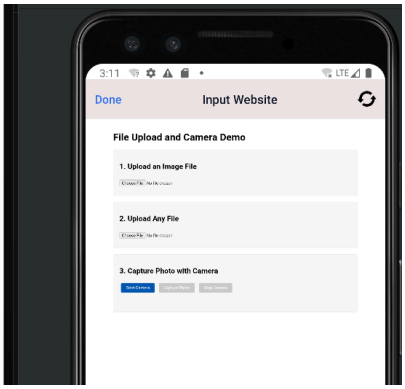
Effect:
Attackers could use remote webview debugging to reveal hidden files.

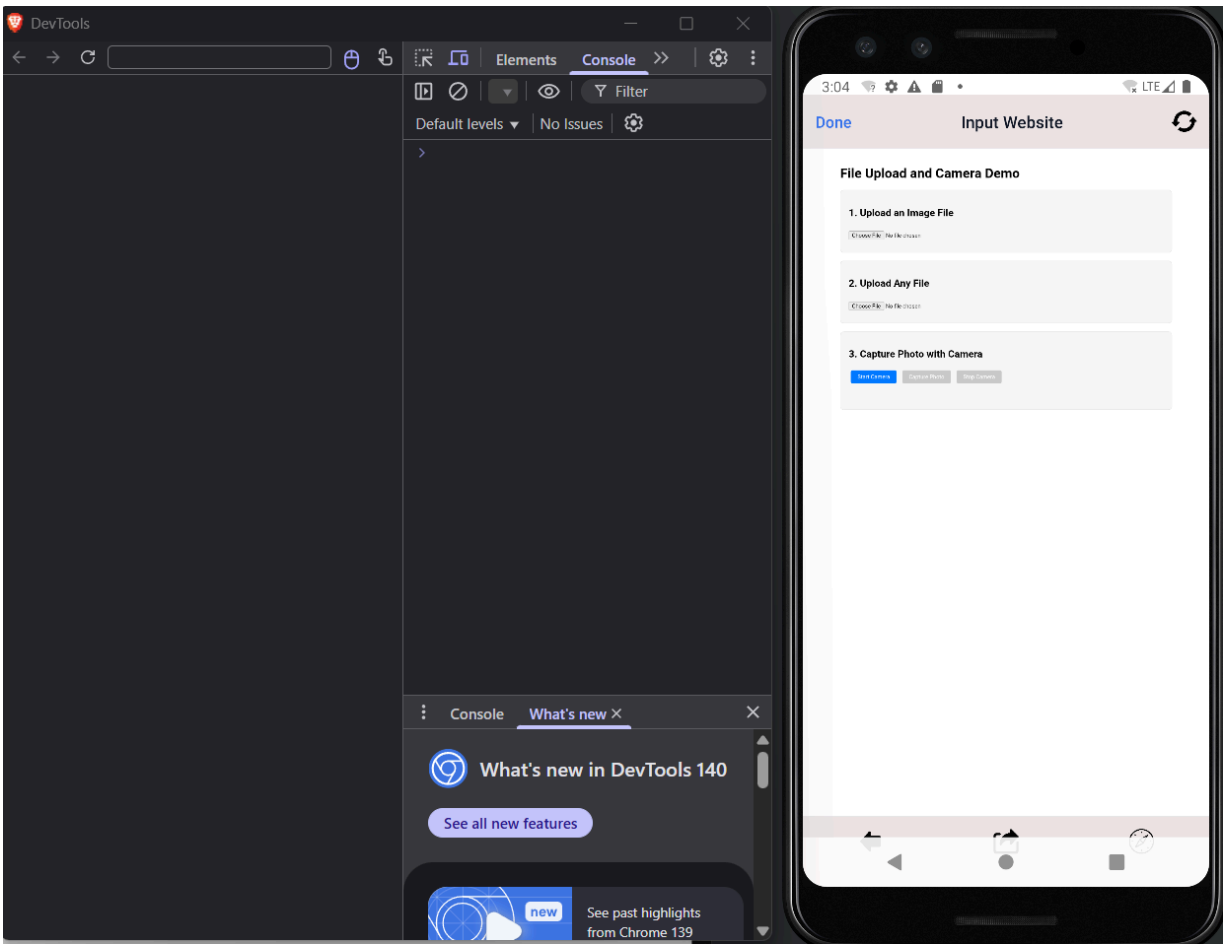
Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

MobSF flagged this issue because the APK has remote WebView debugging enabled. This setting allows developers (or anyone with physical or debugging access to the device) to open chrome://inspect in Chrome and remotely connect to any active WebView within the app. When debugging is enabled, Chrome will show "WebView in <package-name>", and upon clicking "inspect", Chrome DevTools opens, giving full access to the WebView's content and JavaScript environment.

```
}
if (ReactBuildConfig.DEBUG) {
    WebView.setWebContentsDebuggingEnabled(true)
}
```







Why this matters

While this is useful during development, leaving WebView debugging enabled in production poses a serious security risk. The primary concern arises when the app loads local HTML files — for example, from the assets/ directory using a URL like file:///android_asset/index.html. If an attacker gains access to a device running the app (such as through physical access, an emulator, or a compromised environment), they can use DevTools to display hidden files.

References:

- https://developer.chrome.com/docs/devtools/remote-debugging/webviews
- https://slothiesmooth.dev/post/android-webview-vulnerability/

Additional finding:

- Since webview debugging can help attackers for recon, there are actually some ways to enable debugging by making a smali modification to the apk, enabling the webview. (https://slothiesmooth.dev/post/android-webview-vulnerability/)

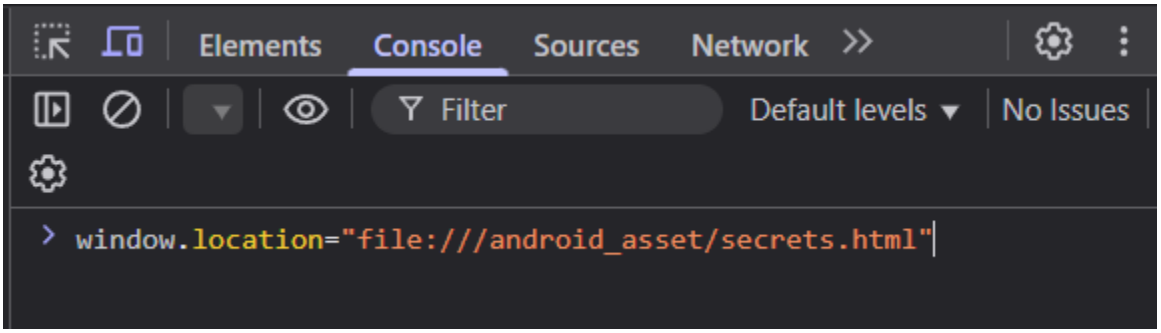
I am going to show that an attacker can display hidden files through WebView remote debugging.

1. I will apply WebView.setWebContentsDebuggingEnabled(true) in the app
2. I will open a webview through the app

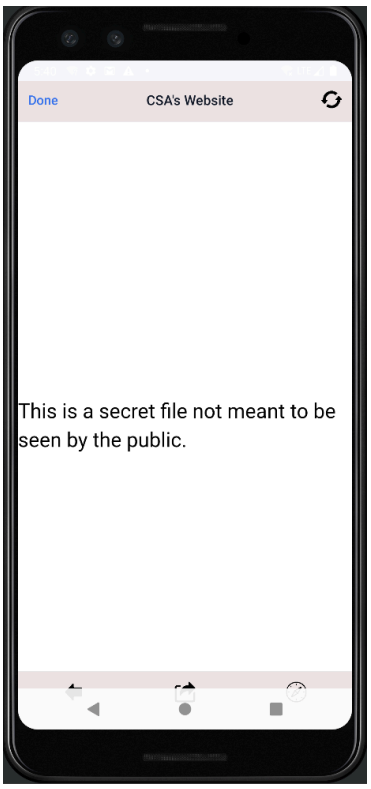
3. I will use chrome://inspect to attach DevTools and use the Console with JavaScript access to the WebView page to run window.location = "file:///android_asset/secrets.html"
4. I expect the WebView to render the html file

Step 1: WebView.setWebContentsDebuggingEnabled(true)
Step 2: Open a webview

Step 3: Use chrome://inspect to run window.location



Step 4: Observe the hidden file rendering on the WebView



Control ID:
US07-MOBSF01-C01. Remove threat by ensuring the app sets WebView Remote Debugging to false.

Control:
Remove threat by ensuring the app sets WebView Remote Debugging to false.

Implemented at:
Client App

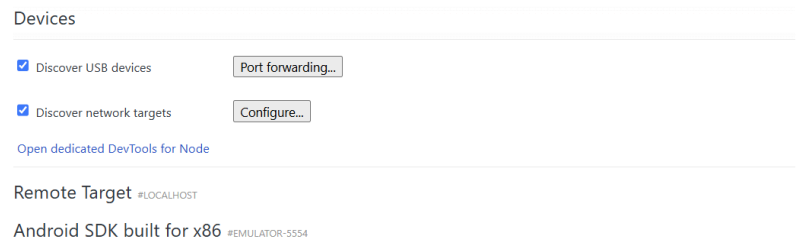
Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

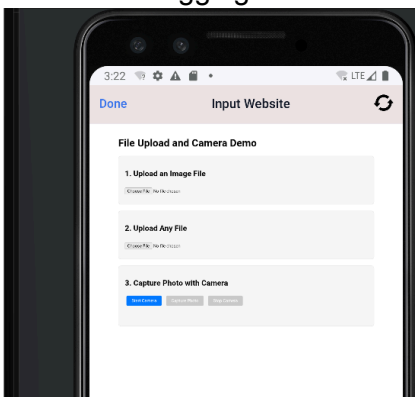
I am going to show that by setting WebView Remote Debugging to false protects exfiltration of hidden files.

1. I will apply WebView.setWebContentsDebuggingEnabled(false) in the app
2. I expect the chrome://inspect to not work

Step 1: WebView.setWebContentsDebuggingEnabled(false)
Step 2: no option to chrome://inspect

As shown below, there is no option to inspect the webview after we set debugging to false.





Status:
tested (tested for effectiveness)

Control ID:
US07-MOBSF01-C02. Remove threat by ensuring the app set allowFileAccessFromFileURLs webview setting to false.

Control:
Remove threat by ensuring the app set allowFileAccessFromFileURLs webview setting to false.

Implemented at:
Client App

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

I am going to show that by setting allowFileAccessFromFileURLs webview to false, prevent attackers from accessing other files within the “files://” directory.

1. I will apply allowFileAccessFromFileURLs to false
2. I expect even if a command runs from the console, the directory cannot be changed

---

Step 1: allowFileAccessFromFileURLs={false}

```
<WebView
    ref={webref}
    // source={{ uri: currentUrl }}
    source={{ uri: 'file:///android_asset/index.html' }}
    javaScriptEnabled={true}  // Enable JavaScript
    sharedCookiesEnabled={true}  // important
    thirdPartyCookiesEnabled={true} // allow cross-site cookies
    allowFileAccess={true} // ANDROID ONLY
    allowFileAccessFromFileURLs={false} // ANDROID ONLY
    onLoadStart={() => setLoading(true)}
    onLoadEnd={() => setLoading(false)}
    onNavigationStateChange={(nav) => {
        setCanGoBack(nav.canGoBack);
        if (nav.url) setCurrentUrl(nav.url);
    }}
    // Keep nav inside the WebView (don't pop Safari)
```

Step 2: Changing directory is not allowed thru the console

```
> window.location="file:///android_asset/secrets.html"
✕ ▶Not allowed to load local resource:                    VM29:1
  file:///android_asset/secrets.html
<· 'file:///android_asset/secrets.html'
>
```

Status:
tested (tested for effectiveness)

Issue: App can read/write to External Storage. Any App can read data written to External Storage.

Issue ID:
US07-MOBSF02. My app writes to external storage. Attackers could read the files created in external storage.

Cause:
My app writes to external storage.

Effect:
Attackers could read the files created in external storage.

Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

MobSF flagged this issue, because the app handles file uploads through a WebView (e.g., when a user takes a photo or video using <input type="file"> in a webpage, it creates a temporary file in the external storage.

```
PS C:\Users\User> adb logcat -s RNCWebViewModule:D
--------- beginning of system
--------- beginning of main
10-01 01:47:56.531 20949 20949 D RNCWebViewModule: 🎭 attempting to create image in external stora
ge public dir: /storage/emulated/0/Android/data/com.ahmadsyuaib.androidmobilebankingapp/files/imag
e-145056577629588233.jpg
10-01 01:47:56.552 20949 20949 D RNCWebViewModule: 🎭 attempting to create image in external stora
ge public dir: /storage/emulated/0/Android/data/com.ahmadsyuaib.androidmobilebankingapp/files/vide
o-997638734222074902.mp4
10-01 01:47:58.316 20949 20949 D RNCWebViewModule: 🎭 attempting to create image in external stora
ge public dir: /storage/emulated/0/Android/data/com.ahmadsyuaib.androidmobilebankingapp/files/imag
e-7589898024196394653.jpg
10-01 01:47:58.318 20949 20949 D RNCWebViewModule: 🎭 attempting to create image in external stora
ge public dir: /storage/emulated/0/Android/data/com.ahmadsyuaib.androidmobilebankingapp/files/vide
o-7786313070821162746.mp4
```

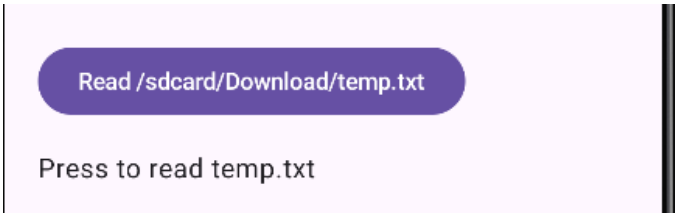
Files:
- com/reactnativecommunity/webview/RNCWebViewModuleImpl.java, line(s) 453
- io/invertase/firebase/utils/ReactNativeFirebaseUtilsModule.java, line(s) 114,123,124,125

<u>Why this matters</u>
Others apps with access to external storage might find and misuse them.

I am going to show that an attacker can locate the temp file and read its contents.

1. I will create a temporary file in the external storage thru the use of a webview that handles file uploads
2. I will use another app with access to the external storage to read the temp file
3. I expect the temp file to be successfully read “this is secret stuff”

Step 1: Create a temporary file

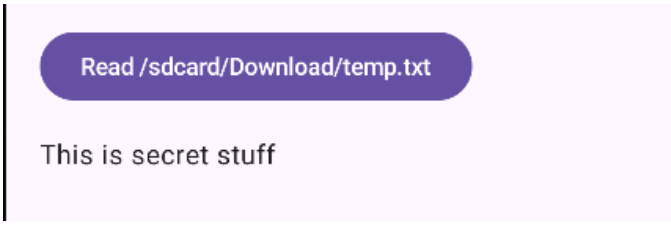


Step 2: Another app with access to external storage

```
val downloadFile =
    File(Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_DOWNLOADS), "temp.txt")
return try {
    if (downloadFile.exists()) {
        val text = downloadFile.readText()
        Log.d(TAG, "Read from /sdcard/Download/temp.txt: $text")
        text
```

Step 3: Read the temp file

Observe that the file created and stored in an external storage, allow other applications to also locate the file and read its contents.



Control ID:
US07-MOBSF02-C01. Remove threat by updating target SDK to Android 11 to enforce scoped storage, disallowing other apps from accessing private files.

Control:
Remove threat by updating target SDK to Android 11 to enforce scoped storage, disallowing other apps from accessing private files.

Implemented at:
Client App

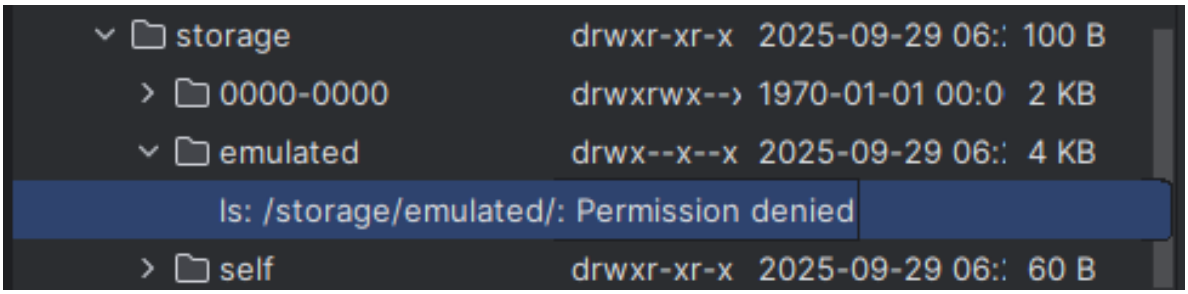
Demonstration:
- Show how the control mitigates the threat.

- Explain with evidence on how you implement the control?

Note: this control's implementation has been shown in User Story 1, 2, 3.

For creation of temporary files, the client app should write it in its own external scoped storage instead. If a file is written to scoped storage, other applications will not be able to view or modify the file created.

Observe that with scoped storage, only the application itself is able to view the files it created.



Status:
tested (tested for effectiveness)